智能运维平台

智能运维 AI 技术原理与工程实现

Kubernetes 开发与架构人员版

文档版本 V7.0

发布日期 2026 年 11 月 1 日

适用范围 非虚拟化智能运维

模型返回检查请求，平台代码执行查询，程序把实际结果交回模型，再形成可复核的分析。

本方案从服务调用、JSON 参数、循环程序、任务状态与资源读取出发，解释模型、提示词、上下文、工具调用、Agent 和 MCP 在产品中的具体工作，并对应项目实现。

阅读顺序

第 1 至 3 章 整体过程 AI 基础概念 模型请求内容

第 4 至 7 章 函数请求 实际执行 Agent 循环 MCP 接口

第 8 至 11 章 Pod 案例 问数计算 规则分工 固定取证

第 12 至 14 章 状态与成本 组件部署 源码导航与技术价值

01 在熟悉的排障程序中增加模型调用

智能运维的执行主体仍然是普通程序。Go 服务管理资源、权限和任务，Python 程序组织调查，数据适配器读取 Kubernetes、指标和日志，数据库保存执行记录。增加 AI 后，程序多了一个步骤：把问题、已查到的数据和可用函数说明发给模型，请模型返回下一项检查请求或分析结果。



图 1 从问题到函数请求 再到查询和分析

从一次函数请求理解整条链路

用户选择一个 Pod，询问“它为什么频繁重启”。平台先固定 Pod UID、命名空间、时间窗口和当前用户权限。Python 程序把这些上下文及只读工具说明交给模型，模型返回“调用 resource_history，参数是该 Pod 和故障窗口”。平台代码执行读取，把结果保存后交回模型。

模型收到真实观测，再提出下一项查询，或者生成对现象的解释。循环由程序驱动，访问后端的是平台工具，最终答案也要由服务端检查。操作者输入实际命令、确认并执行之后，恢复检查另外读取新的资源状态。

本产品中 AI 承担的工作

模型理解人员的问题，把问题映射到已经开放的工具和参数；读取工具返回的数据后，组织解释、诊断假设和后续检查。精确计数、权限判断、资源身份核对、固定规则判断和实际命令执行，由平台对应代码完成。

阅读时抓住三个主体

模型服务负责生成文字或函数调用请求；调查程序负责把模型与工具串成循环；平台工具负责真正读取数据。 后文所有 AI 名词都围绕这三个主体展开，并对应项目中的具体实现。

02 大模型和四个基础概念

在本项目里，大模型是一个通过 HTTP 调用的服务。它接收一组消息及相关参数，使用已经训练得到的模型权重处理输入，再生成文字或结构化结果。这个使用已有权重生成结果的过程叫“推理”。本轮调查请求执行的是推理。

模型权重是训练得到的一组数值参数。模型服务把输入文本拆成 Token，使用这些参数计算并生成输出 Token，再由接口返回文字或函数请求。

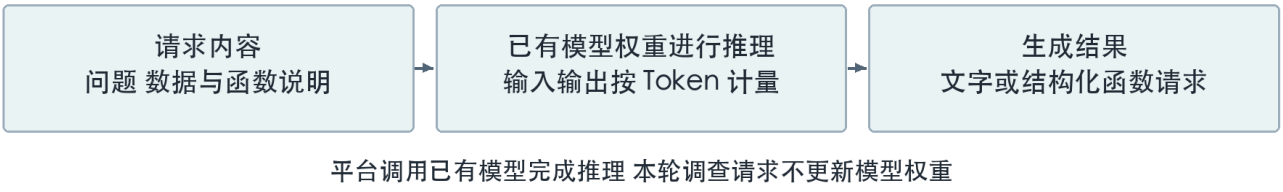


图 2 调用已有模型进行推理

模型处理的是本次收到的输入

模型服务接收平台整理的数据。目标 Pod 的当前状态、历史观测和本轮权限，均通过平台请求提供；读取集群和数据库的代码仍在平台侧。模型基于输入生成结果，平台随后判断这些结果是否符合业务要求。

名词	在这套系统中的实际含义	对开发人员的意义
大模型 LLM	接收输入并生成文字或结构化结果的模型服务	把它看成一个需要超时、认证及返回校验的外部接口
提示词 Prompt	输入中告诉模型要完成什么、遵守什么要求的文字	是请求内容中的指令，程序仍要执行权限和参数校验
上下文 Context	本次模型请求实际带上的指令、问题、历史和工具结果	模型分析本轮问题所使用的输入材料
Token	模型处理文本时使用的片段单位	用于限制输入输出长度、记录用量和控制调用预算

模型配置决定调用哪个服务

平台的 Model Profile 保存模型标识、版本、服务地址、凭据引用、输入长度和输出上限等配置。Python 根据该配置创建模型客户端。HolmesGPT 是调用模型和组织工具执行的 Python 库，实际模型由这个配置指定。

03 第一次发给模型的内容

模型收到的内容比用户输入的一句话多。平台先读取本轮任务上下文，再将固定指令、真实任务绑定、已完成的读取结果和用户问题组织为消息列表。模型还会收到本轮允许使用的函数定义。

请求内容的等价写法

下面用 Python 数据结构说明请求内容的组成。变量来自平台配置和本轮任务，示例省略无关字段；实际模型接口调用由 HolmesGPT 及其 LiteLLM 依赖封装。

```
request = {
    "model": profile.modelID,
    "messages": [
        {"role": "system", "content": policy},
        {"role": "system", "content": serialized_task},
        {"role": "user", "content": committed_results},
        {"role": "user", "content": question},
    ],
    "tools": allowed_tool_definitions,
    "max_tokens": profile.outputTokenLimit,
}
```

每一部分分别解决什么问题

policy 要求只使用开放的只读工具、依据实际读取回答并保留引用。serialized_task 说明本轮任务、对象、范围、时间和预算。committed_results 是平台已经执行并保存的读取结果，question 是本轮问题。tools 列出模型可以申请的函数及其参数说明。

system 和 user 是模型接口中消息的角色字段。本项目将固定要求放入 system 消息，将用户文字、历史及日志等观测材料作为数据传入。平台没有把这些文字转成业务授权，实际权限仍在 Go 服务中核验。

为什么先要准备这些内容

例如“它为什么又重启了”，平台先把“它”落实为已验证的 Pod 身份，并保存固定时间窗口；模型再结合观测理解“重启”的具体情况。对象选择、调查时间和权限来源因此都能在程序中找到对应记录。

04 工具调用是模型返回函数名和参数

这里的“工具”就是平台已经实现的一个只读函数。模型通过函数说明知道可以查什么、需要传什么参数。工具调用也叫 Tool Calling 或 Function Calling，指模型按接口约定返回函数名称和参数，由调用方程序继续处理。

先把函数说明交给模型

resource_history 的作用是读取目标资源在绑定窗口内的历史观测。下面是函数定义的字段节选，用途说明作中文表达；参数校验采用 JSON Schema，与开发 CRD 时声明字段类型和枚举值的思路相通。

```
{
  "type": "function",
  "function": {
    "name": "resource_history",
    "description": "读取资源在绑定窗口内的历史观测",
    "parameters": {
      "type": "object",
      "properties": {
        "canonicalId": {"type": "string"},
        "windowRole": {"enum": ["fault", "baseline", "recovery"]}
      },
      "required": ["canonicalId", "windowRole"],
      "additionalProperties": false
    }
  }
}
```

模型看到的是函数名称、用途和输入格式。Python 和 Go 的函数实现保存在服务端，工具目录按任务类型开放。

模型返回的是一个调用申请

模型返回的 function 字段示例如下。TARGET_ID 在这里代表任务已绑定的完整资源标识。

```
{
  "name": "resource_history",
  "arguments": "{\"canonicalId\":\"TARGET_ID\",\"windowRole\":\"fault\"}"
}
```

Python 解析 arguments 字符串，再执行网关调用。fault 使用服务端固定的故障窗口；模型产生这组字段时，实际查询尚未执行。

05 平台怎样执行并返回查询结果

Python 读取模型返回的函数名和参数，交给项目的 PlatformTool 适配器。适配器先向 Go 服务申请一个 Step，也就是数据库中的一次调用记录；随后通过 MCP 客户端请求 Go 工具网关执行该函数。

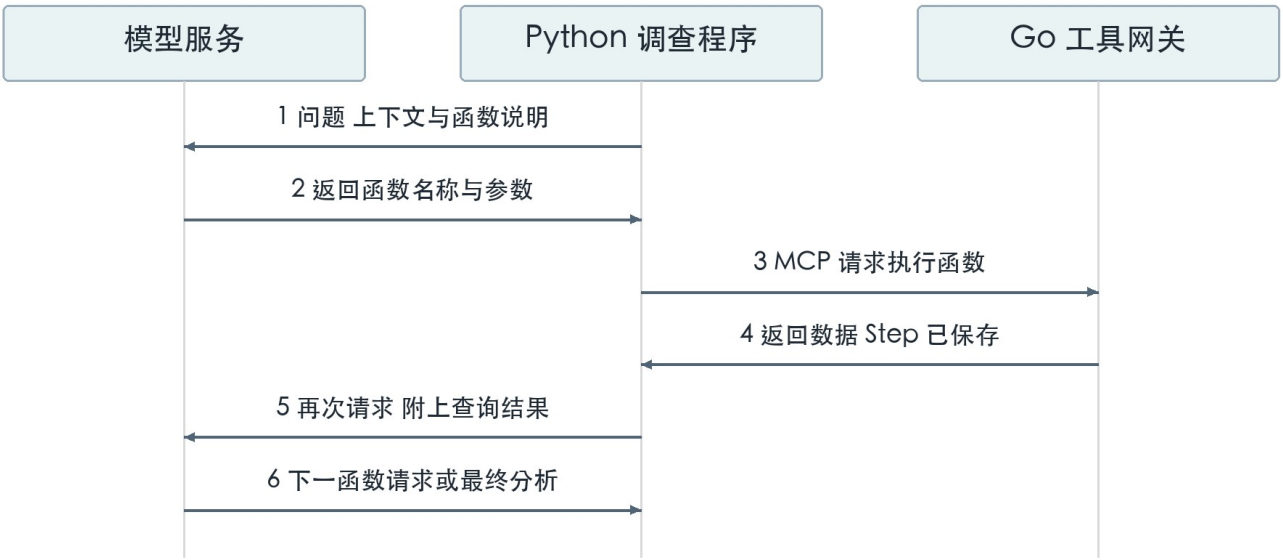


图 3 一次工具读取前后的两次模型请求

从模型字段到程序调用

下面展示模型返回字段的处理方式。target_id 在示意中代表平台已绑定的完整资源标识；真实调用携带该对象的实际 Canonical ID。

```
name = tool_call.function.name
args = json.loads(tool_call.function.arguments)
# name == "resource_history"
# args == {"canonicalId": target_id, "windowRole": "fault"}

allocation = job_api.allocate(name, args)
result = bridge.call(name, args, allocation)
```

Go 网关核对服务身份、任务签名、租约、工具目录、参数结构、当前授权、范围和预算，校验通过后调用对应业务函数。resource_history 从平台保存的资源观测中读取历史；当前状态、指标、日志等工具则走各自的业务接口和来源适配。

结果先保存再交回模型

工具返回状态、数据与引用；历史结果包含记录、事件、观测时点和覆盖，projection 保存资源字段。Go 脱敏并校验大小与结构，保存 Step 后，Python 将实际结果交回模型。相同读取按任务及参数摘要复用，失败则保留实际失败状态。

06 Agent 是驱动模型与工具的循环程序

Agent 在这里指一段持续调用模型、解析其请求、执行工具并反馈结果的程序。模型负责生成下一步请求，Agent 程序负责把请求落到调用链。项目复用 HolmesGPT 的 ToolCallingLLM 实现这段循环。

用普通程序看清执行过程

下面是与当前职责分工对应的简化伪代码。循环由 HolmesGPT 实现；模型调用预算、工具执行、结果保存和最终校验由项目适配共同完成。

```
messages = build_messages(job, committed_steps)

for turn in admitted_rounds:
    reply = call_model_with_budget(messages, tools)
    if reply.tool_calls:
        for call in reply.tool_calls:
            step = allocate_or_reuse_step(job, call)
            result = read_committed_or_call_mcp(step, call)
            messages = append_tool_result(messages, call, result)
        continue

    answer = validate_output_and_references(reply.content)
    return commit_answer(job, answer)

finish_with_budget_or_execution_state(job)
```

模型为什么知道下一步可以查什么

第二次模型请求继续带上原有问题和必要上下文，并加入上一轮的函数请求及其实际返回数据。模型据此返回下一项函数请求或最终分析。一次用户提问由此形成多次模型接口调用；程序能检查到每次函数名、参数和结果，工具选择继续经过实际调用校验。

与控制器循环的联系

两者都由程序组织“读取、判断、执行、再读取”。本项目的调查循环使用只读工具，目标是完成取证和分析；修改资源由本人确认的独立处置流程执行。当前 ToolCallingLLM 的一次调用设置 8 步迭代上限，并同时受任务时间、Token、调用次数和结果大小限制。

07 MCP 连接的是调查程序和平台工具

本系统包含两类不同的网络调用。Python 向模型服务请求生成结果；Python 的 MCP 客户端向 Go 工具网关请求执行函数。MCP 是后一类调用使用的协议，负责工具发现、参数传递和结果返回。

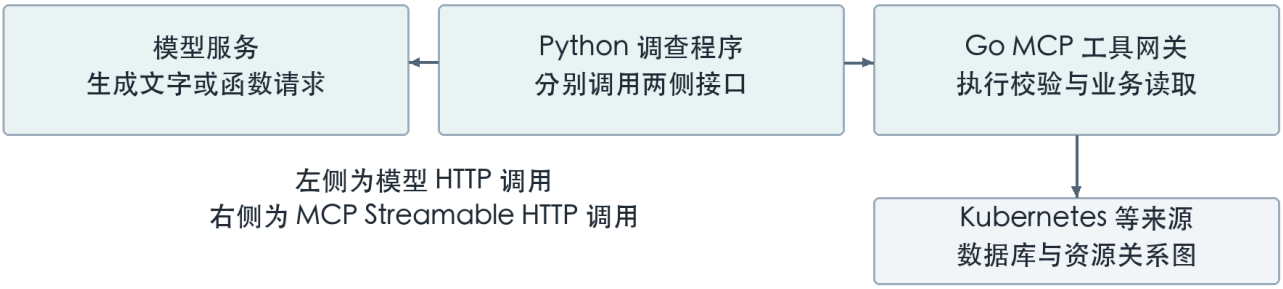


图 4 Python 分别调用模型接口和 MCP 工具接口

调用方向	传递的主要内容	项目采用的技术
Python 到模型服务	消息、函数说明、输出要求及模型返回	HolmesGPT、LiteLLM 与兼容 Chat Completions 的模型接口
Python 到 Go 工具网关	工具名称、参数、任务身份和 Step 信息	MCP SDK 与 Streamable HTTP 传输
Go 到业务数据来源	已授权的资源读取和固定查询	平台业务模块、数据库和 Kubernetes 等来源适配

MCP 在代码中的位置

Python 的 MCPBridge 使用官方 Python SDK，先初始化连接并读取工具目录，再调用 call_tool。Go 网关使用官方 Go SDK 注册工具，实现 tools/list 和 tools/call 等协议处理，并把请求交给业务函数。

项目还在这条链路加入 mTLS 服务身份、签名任务上下文、当前授权及调用预算检查。这些是平台的业务控制，协议传输与业务控制共同构成一次实际调用。

为什么固定取证也使用 MCP

固定取证程序同样需要读取资源、指标和日志，因此复用这套工具接口。它按程序规定的次序调用 MCP，不发起模型请求。由此可以分别理解“是否调用模型”和“通过什么接口执行查询”。

08 一次 Pod 重启问题怎样逐步执行

以下用同一个问题串起模型请求、平台查询和最终解释。用户已选中一个 Pod，调查窗口固定为 02:00 至 02:15；下表中的时间和数值用于演示字段含义。

观测字段	02:00 的记录	02:10 的记录
Pod UID 与容器	同一 UID，容器 app	同一 UID，容器 app
restartCount	4	7
lastState.terminated	本例省略	reason 为 OOMKilled，finishedAt 为 02:09

第一步准备任务与允许使用的函数

Go 保存目标 UID、范围、绝对窗口及任务记录。Python 先通过 get_assistant_context 取得平台上下文，再组织模型输入。任务规定的必要读取由程序预先完成；后续模型可用的函数按本轮工具目录提供。

第二步执行模型提出的读取

模型根据“为什么频繁重启”及工具说明，提出 resource_history 请求。平台实际读取并保存类似表中的历史字段，返回同一对象在窗口内的观测。后续读取仍由本轮模型请求或任务的固定步骤决定；这份示例说明一条调查路径。

第三步用实际事实形成解释

模型继续申请 evaluate_fault_chain，参数引用已经成功提交的 Step。Go 的有限规则核对同一 UID 的容器重启计数是否上升，以及终止时间是否落在故障窗口、退出原因是否为 OOMKilled。模型收到这些评估结果，再解释已观察事实，并组织检查内存时序、资源配置和日志等后续事项。

第四步区分已观察事实和待检查原因

这组输入支持说明“故障窗口内容器发生重启，终止记录报告 OOMKilled”。Node 内存耗尽、容器限制、应用内存泄漏属于进一步核验的原因。答案保存事实引用与后续检查，平台按实际成功步骤校验，新的检查再产生新的记录。

09 问数问题交给固定查询代码计算

问数使用同一套模型与工具机制，但数据处理目标是得到可核对的统计结果。例如“按健康状态统计 Pod”，模型先读取数据集定义，再用固定格式表达“查什么、按什么分组、筛选什么”。这个对象叫 QueryIntent，即查询意图。

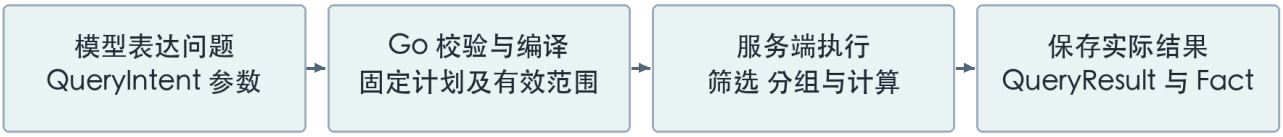


图 5 自然语言意图交给 Go 固定查询代码处理

QueryIntent 的实际字段

```
{
  "schemaVersion": "query-intent/v1",
  "datasetId": "resources.current",
  "operation": "aggregate",
  "metrics": ["resource_count"],
  "dimensions": ["health"],
  "filters": [{"field": "kind", "operator": "eq", "value": "Pod"}],
  "sort": [],
  "limit": 10
}
```

模型将该意图交给 execute_analysis_query。Go 检查数据集和算子是否开放、字段是否合法，再叠加服务端的租户、权限及页面范围，选择固定查询计划。resources.current 在资源图的实际授权快照上统计；指标、日志和网络数据集采用各自模板。

数值由服务端产生

平台完成计数、分组、比较和单位处理，保存查询结果及数值单元的标识。查询结果叫 QueryResult，里面可引用的数值单元叫 Fact。模型的回答引用这些标识，页面从已保存结果展示实际数值。

这与工程中“请求 DTO、参数校验、查询计划、执行、返回结果”的流程对应。模型把自然语言转成 DTO，服务器继续控制数据范围和计算方式。

10 模型解释与诊断规则怎样协同

规则代码与模型承担两类工作。规则检查明确的字段和条件；模型结合多个已取得的结果解释问题、提出假设和后续检查。它们可以处理同一个问题，但输出的依据和职责分别保存。

组件或机制	实际检查的内容	在分析中的作用
K8sGPT 原生 Analyzer	固定参数读取 Pod、Node、PVC 的原生异常	提供检查结果，本项目使用其 Analyzer 路径
NPD 规则与 Coroot Check	内核日志规则匹配、确定数值与阈值比较	提供有具体条件的异常事实
资源关系图与正式配方	真实关系、必要事实、来源和排除条件	确定检查对象并按适用规则形成正式评估
大模型	问题、工具结果、规则评估及历史	解释事实、组织诊断假设和下一项检查

K8sGPT 在这里执行原生 Analyzer，不启用它的模型解释功能；本平台的模型工具循环使用 HolmesGPT，二者承担不同职责。

资源关系图提供实际对象关系

Ariadne 与 Ontology 适配组织 Pod、Node、PVC、StorageClass 和硬件等实体及关系。模型通过平台工具读取相关对象和路径，随后围绕这些实际对象继续检查。资源关系来自引用、选择器、调度和可信映射等平台数据。

具体规则如何使用这些事实

PVC 的正式配方核对 Pending 状态、与实际 CSI 驱动对应的错误事件及真实供应路径，并检查时效、独立证据和反证。正式 RCA 即正式根因分析，由独立的配方评估和提交流程保存版本。助手的故障链工具只评估观察与关系，不能直接确认正式 RCA。

引用校验保障什么

服务端核对答案中的资源、查询结果和证据标识是否来自本轮成功步骤或重新授权的历史。它保障依据真实可追溯；模型解释中的诊断假设继续保留支持、反证和待检查项，正式规则按自身条件判定。

11 哪些调查步骤按固定程序执行

并非所有调查任务都让模型决定下一步。项目为明确提交的故障链请求设置了固定取证模式：Go 将该任务的模型请求次数和输入输出 Token 预算设为零，Python 执行预定的读取及有限规则评估。

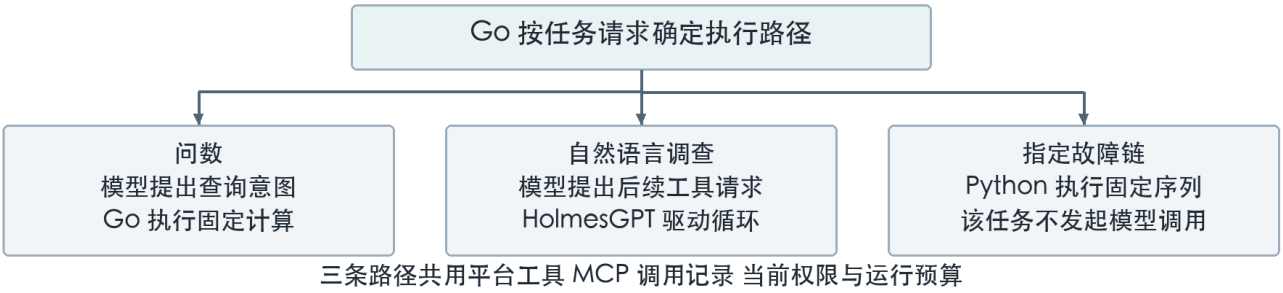


图 6 问数 模型调查与固定取证的执行分流

固定取证路径的等价程序

```
context = call_mcp("get_assistant_context", {})
request = context["faultChainRequest"]

read_target_history_findings_and_changes(request)
read_admitted_relations_and_applicable_datasets(request)
evaluation = call_mcp("evaluate_fault_chain", {"stepIds": step_ids})
return build_answer_from_fixed_evaluation(evaluation)
```

这是职责示意伪代码。实际读取有资源类型、来源能力和预算限制；resource_history、resource_findings、resource_changes 以及适用的关系和数据查询都经过同一工具记录与授权流程。

三种工作分别在哪里发生

问数让模型提出 QueryIntent，由 Go 完成固定计算。自然语言调查使用 HolmesGPT 循环，由模型提出后续工具请求。指定故障链任务执行固定取证序列，按程序组织观察和缺口；后续自然语言追问使用已经核验的历史继续分析。

固定取证仍遵循任务准入、模型配置等运行要求；“零模型调用”说明这类任务的执行路径与用量。它与模型调用失败后的处理属于不同流程。

12 任务状态与上下文怎样保存

调查的持续状态保存在 PostgreSQL，后续模型请求由 Python 重新组织输入。理解这些持久对象，可以看清页面断开、Worker 接管、连续追问和模型输入长度各自如何处理。

持久对象	保存什么	与模型调用的关系
会话与轮次	范围、问题、历史及本轮绑定	为下一次请求重新准备输入
调查 Job	状态、预算、租约和执行期限	驱动一次后台调查任务
Step	一次模型调用或工具调用的状态与结果	支撑已完成读取复用和后续事实引用
分析与引用	回答、来源、时间及对应版本	保留每轮实际依据，追问时重新授权

调查 Job 与 Kubernetes Job 的区别

调查 Job 是平台数据库里的业务任务，由 Go Worker 领取并推进，Python Investigator 是调查运行时。命令处置时创建的 Command Runner 才采用一次性 Kubernetes Job。两个 Job 的职责和存储位置明确分开。

Token 和预算控制请求规模

模型服务有本次输入输出长度上限。平台限制模型请求数、输入输出 Token、工具调用、时间和返回大小；BudgetedLLM 在调用前预留用量，取得结果后结算。运行时将已提交结果整理为模型输入，较早历史按服务端规则压缩并保留关键引用。

持久化历史与训练模型是两件事

历史写入数据库后，下轮再读取并放入上下文，这个过程没有修改模型权重。训练用于产生或更新模型权重；本调查链路使用已有模型做推理。常说的 RAG 指先检索材料再提供给模型；本主链路通过授权工具补充运行事实，向量检索和模型微调不参与上述调用过程。

13 技术组件与部署关系

平台按业务服务、调查运行时、模型服务和数据来源组织部署。模型接口及工具接口在图中分别标出，调查程序同时连接两侧，Go 服务持续掌握任务状态与数据访问权限。

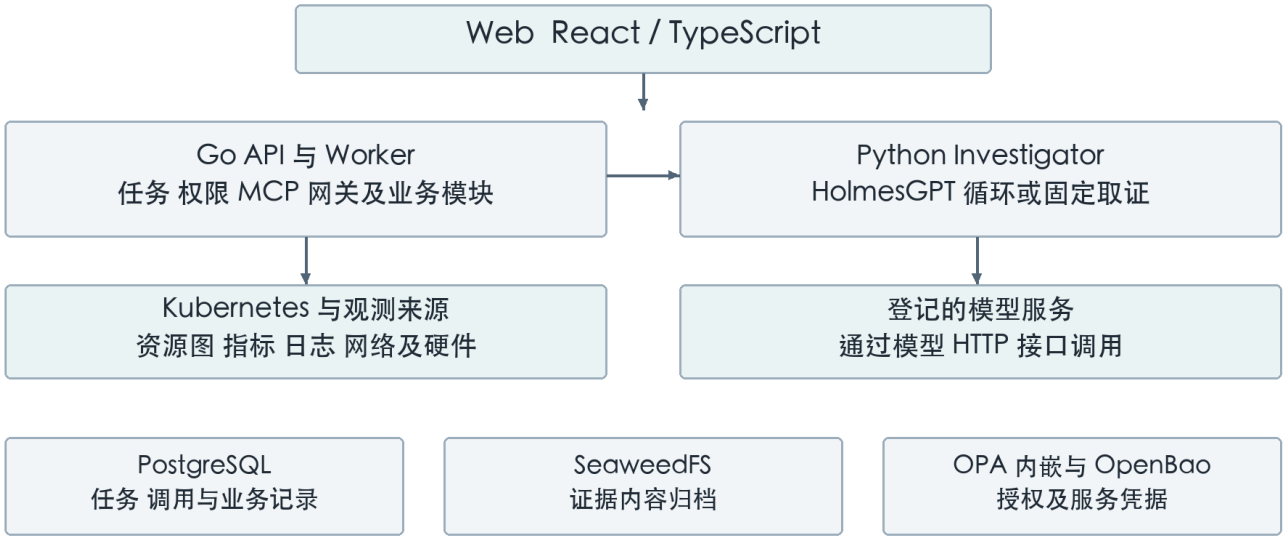


图 7 平台进程 模型端点和数据来源的部署职责

AI 调用相关组件

Python 运行 Investigator。HolmesGPT 0.42.0 组织循环，LiteLLM 发送模型请求，BudgetedLLM 记录调用并控制预算；MCP Python SDK 与 Go SDK 连接调查程序和平台工具。

开发模板以 Ollama 接入自托管模型；代码也支持限定的 DeepSeek API 路径。任务从获准 Profile 读取实际模型标识、版本及端点。

数据与资源相关组件

Go API 和 Worker 共享业务模块，Ariadne 与 Ontology 适配提供资源关系。Kubernetes API 提供原生状态，VictoriaMetrics 提供指标，VictoriaLogs 提供日志，DeepFlow 提供已接入的通信观测，Redfish 与 gofish 提供硬件读取。PostgreSQL 保存任务、观测及业务记录，SeaweedFS 归档调查或处置所需的证据内容。

权限与处置相关组件

OPA SDK 内嵌在 Go 中执行授权，OpenBao 管理证书、密钥及短期凭据。每次工具读取都核验当前权限。操作者输入并确认实际命令，隔离 Runner 经 kubectl 或 Ansible 通道执行，完成后读取目标及关联状态进行恢复验证。

14 源码阅读导航与产品技术价值

在工程中定位这条链路，可以先看 Python 如何组装请求与接收模型结果，再沿 MCP 客户端进入 Go 网关，最后查看具体查询和规则实现。下表文件名按所在目录分组说明。

Python 调查运行时

目录为 services/investigator/src/investigator。以下入口能对应到本方案中的请求、循环、预算和两种执行路径。

文件	主要入口	对应的工作
worker.py	调查请求处理与 executionMode 分流	选择固定取证或模型调查
assistant_binding.py	messages 与输出引用约束	组织提示词、任务数据、历史及问题
holmes_adapter.py	investigate 与 PlatformTool	Holmes 循环、工具定义和调用适配
budget.py	BudgetedLLM.completion	模型请求记录、预算预留及结算
mcp_bridge.py	catalog 与 call	通过 MCP 发现和调用 Go 工具
fault_chain.py	collect	零模型调用的固定取证路径

Go 服务中的对应实现

internal/investigation/mcp/server.go 注册工具并校验请求，随后调用业务接口；internal/investigation/tools/catalog.go 决定每类任务开放哪些函数。internal/analytics/compiler.go 负责资源统计意图的计划编译，各来源另有固定查询适配；internal/rca/engine.go 与 repository.go 负责正式规则评估和版本提交。

产品优势落实为具体工程能力

模型把自然语言接到平台已有工具，调查循环使新数据持续参与分析，固定规则保留专业判断条件，数据库记录使调用和引用能够复核。平台以身份、范围、预算和本人确认约束实际读取及修改，将语言交互、专业检查和运维执行连接为可管理的工作过程。